Kingdom of Saudi Arabia
Ministry of Education
Umm Al-Qura University
031

المملكة العربية السعودية
وزارة التعليم
جامعة أم القرى
٠٣١

AI320IT – Machine Learning

Smart Team Task Allocation System Project

Student Name	ID	Section
Refal ZainAlden	445007410	1
Amareej Alzhrani	445007479	1
Huda Alaa Sleem	44511003	1
Zahraa Alsatif	445013956	1
Hajer Sayah	445013975	1

Supervisor:

Dr. Afaf Al-Mehmadi

Table of Contents

Abstract …….…… 3
Introduction …… 4
Problem Definition …….. …… 5
Intelligent Task Scheduling and Fatigue Modeling…….....6-7
AI-Based Task Management and Priority Prediction…….…….. .. 8-9
Task allocation with load management in multi-agent teams……... 10-11
Multi-agent graph neural network-based efficient task allocation for autonomous vehicles with deep reinforcement learning…… 12-13-14
Fatigue Estimation During the Sit-to-Stand Exercise Using Machine Learning…….….....15-16
Detecting Fatigue in Repetitive Physical Tasks Using Multi-Sensor Machine Learning Models……..…….…..17-18
ML-Based Role Prediction for Task Allocation……..19-20
Learning Task Requirements and Agent Capabilities for Multi-Agent Task Allocation……21-22
Task Allocation in Teams as a Multi-Armed Bandit…….……23
Task Allocation in Human–Machine Manufacturing Systems Using Deep Reinforcement Learning…….…… …….. 24
Table (1) Comparison of research …….……...25
Table (2) Comparison of research …….……...26
Detailed ML Module ……27
Comparison with existing research……28
Figure 1 flowchart workflow .....……29
Figure 2 First 5 rows of the dataset & Figure 3 histograms for numerical features……..30
Figure 4 Feature Correlation Heatmap ……31
Figure 5,6 Target Class Mapping ……..32
Figure 7 Random Forest performance report ……33
Figure 8 Random Forest Feature Importances…… 34
Figure 9 Logistic Regression Results (Improved)& 10 Random Forest Confusion Matrix …….35
Figure 11 Logistic Regression Confusion Matrix ……..36
Figure 12 random Forest Predicted Class Distribution ..……..37
Figure 13 Logistic Regression Predicted Class Distribution & Conclusion ….……. ..38
Conclusion …….....39
References ……40-41

Abstract

The project aims to design a machine learning model that improves task allocation among team members through an intelligent system that analyzes individual skills, current workload, task type, priority level, and personal preferences. By incorporating these factors, the system can accurately identify the most suitable member for each task.

Traditional task distribution methods suffer from several issues, including manual assignments, unbalanced workloads, human bias, and difficulty predicting performance quality. Our proposed system applies machine learning algorithms—Random Forest and Logistic Regression—to address these challenges. The results showed that Random Forest significantly outperformed Logistic Regression, demonstrating its strength in predicting the optimal agent for each task.

This report reviews current methods in the field, highlights existing gaps, and presents a model capable of addressing these limitations effectively. The overall goal is to enhance productivity, improve decision-making, and ensure fair and efficient task distribution among team members.
Keywords: Machine Learning, Task Allocation, Productivity, AI-Based Systems, Random Forests, Logistics Regression.

Introduction

This system aims to provide an intelligent and reliable mechanism for distributing tasks among different agents within a shared operational environment. These agents include both human team members and robots with specific functional capabilities. The system analyzes the requirements of each task, evaluates the abilities, workload levels, and availability of every agent, and then assigns the task to the most suitable human or robot. Through this approach, the system reduces reliance on manual planning, minimizes human errors, and supports a more organized and well-coordinated workflow.

The system is designed to assist project managers, supervisors, human team members, and robots that participate in executing tasks within environments where daily collaboration between humans and robots is essential. Its importance becomes evident in settings that require rapid decision-making, managing multiple workloads, and maintaining effective coordination across different agents.

Despite the value of traditional task allocation methods, they suffer from several limitations, including the time-consuming nature of manual planning, the inability to achieve precise workload balance, difficulty tracking sudden changes in agent availability, and weak coordination between humans and robots. These limitations often lead to delays, increased pressure on certain agents, and duplicated or inconsistent work. Such challenges emphasize the need for a smart, data-driven allocation system capable of supporting real-time decision-making.

Integrating machine learning significantly enhances decision quality, enabling the system to analyze current data and learn from previous outcomes. ML techniques help identify the most suitable agent for each task, maintain balanced workloads, and adapt quickly to rapid changes such as overload or agent unavailability—without interrupting the workflow. Over time, the system becomes more efficient and reliable as it continuously learns from actual performance.

This scenario is increasingly relevant as human–robot collaboration expands across modern industries. Socially, the system promotes fairness, reduces stress, and improves overall working conditions. Academically, it provides a valuable framework for studying intelligent coordination and multi-agent systems. Industrially, it boosts productivity in fields such as manufacturing, logistics, project management, and smart services by optimizing resource utilization and reducing execution delays. Overall, the system addresses a growing need for an effective, unified framework capable of managing tasks intelligently within hybrid human–robot environments.

Problem Definition

Brief Description of the Problem

The problem covered in this project revolves around the limitation found in classic task allocation methods, which are still heavily dependent on manual processes, human judgment, and fixed rules. In many real-world environments—especially those where humans and robots work together—task distribution becomes increasingly complex. Managers often struggle to accurately evaluate each agent's skills, workload, capabilities, and availability, which leads to unbalanced assignments, delays, and inconsistencies in task quality. These challenges point out the need for an intelligent system that can analyze different factors and make reliable, data-driven decisions.

Motivation

The development of this system is driven by the growing demand for smarter coordination mechanisms in hybrid environments involving both humans and robots. As tasks become more dynamic and environments more unpredictable, traditional manual assignment approaches quickly show their limitations. They are prone to human bias, require continuous supervision, and fail to adapt to sudden changes such as workload fluctuations or agent unavailability. Machine learning provides an effective solution by enabling the system to learn from previous task outcomes, identify hidden patterns, and generate real-time decisions that improve fairness, balance workload, and reduce errors.

Early Idea of the Suitable ML Approach

To handle this problem in an effective way, the project considers using the supervised machine learning techniques capable of predicting the most suitable agent for each task based on input features. Models such as random forests, logistic regression, decision trees, support vector machines, and neural networks are well suited for analyzing task attributes, performance history, and workload conditions as well. These models can classify and rank agents according to their compatibility with specific tasks, enabling accurate and adaptive task assignments. As the system acquires more data, it becomes increasingly precise and could eventually incorporate reinforcement learning for further improvement based on real-time feedback.

Summary of Research on Intelligent Task Scheduling and Fatigue Modeling

The research by Bogdan Walek and Vilém Novák (2023) [1] talks about an intelligent task planning system that uses Fuzzy Natural Logic. The main research question how to make a system that organizes tasks using words like "very important task" or "less important task" and can adjust to what the user likes over time. Instead of using real people as participants, the system was tested with real task data and user performances in a web app.

The data they used included:

- Task description in words, like how important or urgent a task is.
- User Choices for tasks and reminders.
- Calendar information to plan tasks and meetings.

The system did not use regular machine learning. It used a fuzzy logic system with a modular that learns what the user likes, so it gets better over time.

The main results showed that the system can:

- Sort task well, showing the most important tasks first. (Quote from paper "The system shows the most important tasks first and then the less important ones in a determined ordering.").
- Adjust to user preferences over times. (Quote from paper "Important part of this system is module for learning user preferences in area of task and reminders of tasks.").
- Work in real life as a web app.

But the system has some weaknesses:

- Doesn't work well for all situations, because it uses fuzzy logic and not advanced machine learning.
- Takes time to learn what the user likes, especially at the beginning.
- No strong comparison with modern ML-based task planners.
- Not clear if it can handle lots of tasks or users.

How Our Project can improve this system

Our project is about giving tasks to the best team members based on skills, pest performance, and task difficulty. Compared to the fuzzy system in the paper, our project can improve in this way:

1. Better task-member matching: The fuzzy system mostly orders tasks for one user, but our system can give tasks to the team member who fits best using skills and performance data.
2. Machine Learning Models: We can use Logistic Regression, Random forest, Gradient Boosting, or Collaborative Filtering to guess the best task assignment. These models learn from data and adjust faster than fuzzy rules alone.

3. Simple way to handle task difficulty: Since our project is easier (classifying or recommending tasks), we can mix ML predictions with simple rules to keep fairness, priority, and balance.
4. Better for bigger teams: ML lets the system handle more team members and more tasks, which was a problem for the original system.
5. Mixing fuzzy logic with ML: we can use fuzzy logic for task importance and ML for choosing the best member, so the system is both intelligent like a human and data-based.

Summary of Research on AI-Based Task Management and Priority Prediction

The research by Manazir Musthafa and Dr.Amj Muthukumaran (2024) [2] explains a task management system that uses AI to automatically prioritize tasks based on past data and their information the user enters. The main research question was how to build on intelligent system that can sort task by urgency, deadlines, and importance, while also learning from previous user tasks.

The system might not include human participants is an experiment. Instead, it was tested using historical task data, meaning the system learned from old task records and user inputs inside the application.

The data used in research included simple task details such as:

- The task name
- The deadline
- How urgent the user says the task is
- Past task priorities
- User feedback that helps improve future recommendations

For the machine learning part, the system used several algorithms. They used Random Forest to predict task priority based on the task features. They also used Linear Regression to estimate things related to time and workload. Another method they used was K-Nearest Neighbors (KNN) as part of collaborative filtering to give more personalized suggestions for users. All these models helped the system learn patterns and improve prioritization over time.

The key findings showed that the system can automatically sort and prioritize tasks using AI. One important quote from the paper says "The system uses AI-based engine to prioritize tasks using historical data and real time parameters." The researchers also noted that the interface is simple and lets users enter tasks easily by adding the task name, deadline, and urgency level. The system also supports importing a large number of tasks, which help make the system more scalable. They also mentioned that the system includes multi-user authentication, meaning every user has a private and secure account.

However, the system still has some weaknesses. The paper does not include a deep experimental evaluation with many users, so it is hard to know how well the system performs in real daily tasks. The quality of the AI predictions also depends a lot on the quality and amount of historical data. If there is not enough data, the task recommendations may not be accurate. Another weakness is that the system may struggle if the number of tasks or users becomes very large, since the paper did not clearly show how it performs under heavy load.

How Our Project Improve this System

Our project focused on assessing tasks to different team members based on their skills, past performance, and the difficulty of the task. Because of that, our project can add several improvements to the system in the paper:

1. Better task-to-member matching: The IJERT system only prioritizes tasks for one user, but our project can decide which team member is the best choice for each task using skills and performance data.
2. Using more flexible ML models: We can still use models like Random Forest or KNN, but we can also add other like Logistic Regression or Gradient Boosting to improve accuracy. These models can learn faster than fuzzy rules or simple prioritization.
3. Handling task difficulty more clearly: Since our project includes difficulty levels, we can mix ML predictions with simple rules to keep fairness and balance in the team.
4. Better scalability: Because our project focuses on teams, we can design it to handle many members and many tasks more efficiently.
5. Combining prioritization + assignment: We can take the idea from the paper (AI prioritization) and combine it with automatic task distribution, creating a stronger intelligent system that works for teamwork, not just individuals.

Summary of Research on Task allocation with load management in multi-agent teams

The research by **Haochen Wu, Amin Ghadami, Alparslan Emrah Bayrak, Jonathon M. Smereka, and Bogdan I. Epureanu (2022)** [9] talks about a load-aware task allocation system for multi-agent teams using **Decentralized Reinforcement Learning (DRL)**.
The main research question is how to make multi-agent teams distribute tasks efficiently *while also managing agent load*, preventing overload, saving resources, and keeping the system resilient when unexpected events happen.
Instead of using real people as participants, the study tested the system in **simulated environments**, including:

- Homogeneous robotic agents
- Heterogeneous human–autonomy team simulations

The data they used included:

- Agent load levels during tasks
- Sequences of incoming tasks
- Resource usage patterns
- System behavior in normal and overloaded conditions

The system used **Decentralized Reinforcement Learning (DRL)**, not supervised ML.
Each agent learns from its own experiences, with a reward function that considers:

- How efficiently tasks are handled
- Penalties when an agent becomes overloaded
- Incentives for idling to save resources

The main results showed that the system can:

- Distribute tasks in a balanced way by avoiding overload.
- Make agents decide to decline tasks or stay idle when needed.
- Adapt flexibly to dynamic and unpredictable environments.
- Improve resilience using an "agent importance" metric that estimates how well a team can handle overload.
- Improve resource use and overall team performance.

But the system has some weaknesses:
One limitation is that the system was tested only in simulation and not on real robots or real human–robot teams, which makes its real-world performance uncertain.
Reinforcement learning also requires high computational power and is very sensitive to how the reward function is designed, which can make training difficult.

The model may struggle to scale effectively to very large teams, since coordination becomes harder as the number of agents increases. In addition, the study did not

thoroughly test communication limits or real-world uncertainty, such as network delays or signal loss.

Finally, the system provides only minimal exploration of true human–robot collaboration, so its effectiveness in mixed teams remains unclear.

How Our Project Can Improve This System

Our project is about assigning tasks to the best team members based on **skills, past performance, and task difficulty**. Compared to the DRL system in this paper, our project can improve in the following ways:

1. Better human–team matching

The DRL system focuses on load balancing between agents, but our system can match tasks to the *most suitable team member* using skills, preferences, and historical performance.

2. Using simpler and interpretable ML models

Our project can use Logistic Regression, Random Forest, Gradient Boosting, or Collaborative Filtering to predict the best team-member assignment.

These models train faster and are easier to tune than DRL, which solves one of the weaknesses in the paper.

3. Handling real-world constraints more easily

Our project deals with smaller, more structured data (skills, difficulty, performance), making it easier to incorporate fairness, priority, and team balance rules—without complex RL reward engineering.

4. Better scalability for team settings

Traditional ML models scale better with large teams and many tasks compared to decentralized RL, which becomes expensive as the number of agents grows.

5. Hybrid intelligent decision-making

We can mix rule-based logic (like fuzzy importance rules) with ML predictions to choose the best member—getting the best of both worlds:

- Human-like reasoning (rules / fuzzy logic)
- Data-driven accuracy (ML models)

Summary of Research on Magnnet: Multi-agent graph neural network-based efficient task allocation for autonomous vehicles with deep reinforcement learning

The research by **Lavanya Ratnabala, Aleksey Fedoseev, Robinroy Peter, and Dzmitry Tsetserukou (2025)** [10] talks about a multi-agent task allocation system that uses **Graph Neural Networks (GNNs)** combined with **Deep Reinforcement Learning**.
The main research question is how to make heterogeneous autonomous agents—like UAVs and UGVs—perform efficient decentralized task allocation even when communication is limited, using a CTDE (Centralized Training, Decentralized Execution) framework.

The study does not use human participants.
Instead, it was tested in a **3D simulation environment** with:

- Unmanned Aerial Vehicles (UAVs)
- Unmanned Ground Vehicles (UGVs)

moving under communication and coordination constraints.

The data they used included:

- Agent locations and movements in a 3D grid
- Task locations and dynamically created tasks
- Graph-structured relationships between agents (for GNN processing)
- Travel cost estimations using A* and R* planners
- Conflict detection information for multi-agent coordination
- Performance metrics comparing centralized vs decentralized methods

The system used multiple advanced ML components, including:

- Graph Neural Networks to understand spatial and interaction patterns
- Multi-Agent Deep Reinforcement Learning
- Proximal Policy Optimization (PPO) for training the agents
- CTDE for centralized training but decentralized execution
- Path planning algorithms (A* and R*) for routing decisions

So overall, the framework combines **deep learning + reinforcement learning + graph representation learning**.

The main results showed that the system can:

- Reach **92.5% conflict-free** task allocation success.
- Perform almost like the optimal centralized Hungarian method (only 7.49% difference).
- Outperform decentralized greedy heuristics by a large margin.
- Scale to 20 agents with fast decision times (~2.8 seconds).

- Handle dynamically arriving tasks in real time.
- Remove the need for a central controller while staying near-optimal.
- Work effectively for both UAVs and UGVs in complex environments.

But the system has some weaknesses:

The system was tested only in simulation, not on real robots or under real-world communication conditions, so its actual performance outside controlled environments is uncertain. The combination of GNNs and multi-agent deep reinforcement learning also creates a heavy computational load, which may run slowly on real UAVs or UGVs with limited processing power.

Scalability is another concern, as the model was evaluated only up to 20 agents, leaving its ability to handle larger teams unclear. The approach also relies on stable communication, which is unrealistic in real settings where signal loss or delays could disrupt coordination.

Additionally, path planners like A* and R* may become slow in large or cluttered 3D environments, reducing responsiveness. Finally, the system does not support human–robot collaboration, limiting its applicability to mixed teams.

How Our Project Can Improve This System:

Our project is simpler and more focused on assigning tasks to team members based on **skills, task difficulty, and past performance**, so it can improve or simplify some of the limitations :

1. Simpler and faster decision-making

The MAGNNET system uses heavy GNN + MARL models that need huge computation.
Our system can rely on lighter ML models (Logistic Regression, Random Forest, Gradient Boosting, Collaborative Filtering) that train faster and can run on normal computers without delays.

2. Better fit for human teams

MAGNNET is for robots (UAVs/UGVs).
Our project is designed for human teams, considering skills, previous work quality, workload, and fairness—things GNN/MARL systems do not handle.

3. Less dependence on communication

Their system assumes stable communication between agents.
Our project does not require constant communication because predictions are centralized and simpler—reducing failure risk.

4. More scalable for large teams

Traditional ML models scale better to large teams than GNN-based MARL, which becomes extremely expensive as agent numbers grow.

5. Can integrate task difficulty + member expertise more clearly

MAGNNET focuses mostly on path cost and conflict avoidance.
Our system directly uses skill data, performance history, and workload—making task assignment more human-realistic.

6. Option to mix ML + simple rule-based logic

We can combine ML predictions with fairness rules, priority rules, or fuzzy logic for better control and interpretability—something MARL alone doesn't provide.

Summary of Research on Fatigue Estimation During the Sit-to-Stand Exercise Using Machine Learning

The research by Aguirre, Pinto-Bernal, Cifuentes, Perdomo, Rueda Díaz, and Múnera (2021) [7] discusses a machine learning-based approach to estimate fatigue levels during physical exercise. The main research question is how to replace subjective fatigue assessments with an objective, automated system that utilizes multi-sensor data to estimate fatigue during the Sit-to-Stand (STS) exercise.

The study involved human participants performing repeated STS movements while data was simultaneously collected from motion sensors and physiological monitors. Instead of relying on self-reported exhaustion alone, the system analyzed physical indicators to classify the user's state.

The data they used included:

- Kinematic and temporal features: 32 distinct features derived from upper- and lower-body motion captured by Kinect sensors.
- Physiological data: Heart-rate signals collected to correlate physical exertion with fatigue levels.
- User performance metrics: Time taken to complete sit-to-stand cycles.

The system tested several machine learning algorithms, including Logistic Regression, k-Nearest Neighbors (kNN), Support Vector Machine (SVM), Decision Trees, and Artificial Neural Networks (ANN).

The main results showed that the system can:

- Achieve high accuracy: The Random Forest model with 60 trees outperformed other classifiers, achieving an accuracy of approximately 82.5%.
- Identify key fatigue indicators: The study found that upper-body motion features had the strongest impact on predicting fatigue compared to lower-body features.
- Classify fatigue levels: The model successfully categorized the user's state into three levels: low, moderate, and high fatigue.

But the system has some weaknesses:

- Controlled Environment: The experiments were conducted in a lab setting, which may not fully reflect real-world conditions.
- Sensor Sensitivity: The Kinect sensors used are sensitive to lighting and background noise, potentially affecting reliability in uncontrolled environments.
- Limited Dataset: The study relied on a relatively small dataset, and no real-time fatigue estimation system was developed for immediate user feedback.

How Our Project Can Improve This System

Our project focuses on assigning tasks to team members based on skills, performance, and current workload. Compared to the fatigue estimation system in this paper, our project can improve in the following ways:

1. Integration with Task Allocation: While the paper focuses only on detecting fatigue, our system uses fatigue data to make decisions—automatically reassigning tasks or preventing assignment to exhausted members.
2. More Flexible ML Models: We can employ advanced models like Gradient Boosting or Deep Learning that may generalize better than the Random Forest model used in the study.
3. Real-Time Application: Unlike the study which analyzed data post-hoc, our system is designed to manage workload in real-time, preventing fatigue before it becomes critical.
4. Broader Context: Our system considers fatigue as just one factor among others (skills, priority, difficulty), providing a more holistic management solution than a standalone fatigue detector.

Summary of Research on Detecting Fatigue in Repetitive Physical Tasks Using Multi-Sensor Machine Learning Models

The research by Liu, Dobbins, D'Souza, and Phuong (2023) [8]
explains a multi-sensor fusion approach to detecting fatigue in industrial-like repetitive tasks. The main research question is whether combining data from different types of sensors (IMU, heart rate, force) can lead to more accurate fatigue detection compared to using single-sensor modalities.

The study was conducted in a simulated environment where participants performed repetitive physical motions similar to manufacturing tasks. The system aimed to build a robust model that could assist in preventing work-related musculoskeletal disorders.

The data they used included:

- Motion data: Accelerometer and gyroscope readings from IMU sensors to track movement patterns.
- Physiological signals: Heart-rate data to monitor cardiovascular stress.
- Force measurements: Pressure data collected during task execution.
- Performance features: Temporal and behavioral metrics related to how the task was performed.

For machine learning, the researchers applied Linear Discriminant Analysis (LDA), k-Nearest Neighbors (kNN), and a Polynomial-Kernel Support Vector Machine (P-SVM).

The main results showed that the system can:

- Deliver superior accuracy: The P-SVM model achieved the highest performance with an accuracy of ~94.15%.
- Benefit from Sensor Fusion: Combining motion, heart rate, and force data significantly improved detection results compared to using any single sensor alone.
- Binary Classification: The system effectively distinguished between "high fatigue" and "low fatigue" states.

But the system has some weaknesses:

- Simulated Setting: The study was performed in a simulation rather than a real factory floor, limiting its verified applicability to actual daily operations.
- Binary Limitation: The system only classified fatigue into two states (High/Low), lacking the granularity (e.g., moderate fatigue) needed for subtle task management.
- Feature Engineering: The approach relied heavily on manually engineered features, which may not generalize well to different types of tasks without expert retuning.

How Our Project Can Improve This System

Our project assesses tasks for team members based on skills, past performance, and difficulty. Therefore, our project can add several improvements to the system in the paper:

1. Proactive Workload Management: Instead of just detecting fatigue when it is "High", our system balances the load dynamically to keep fatigue in the "Low" or "Moderate" zones.
2. Multi-Class Fatigue Handling: We can aim to classify fatigue into multiple levels (Low, Moderate, High), allowing for more nuanced decision-making (e.g., assigning lighter tasks for moderate fatigue) rather than a simple binary stop/go.
3. Scalability to Teams: The research focuses on individual worker fatigue. Our project scales this to a team level, optimizing the collective energy of the group rather than just monitoring one person.
4. Data-Driven Assignment: We combine fatigue data with skill data. A fatigued expert might still be better than a fresh novice for a critical task—our system can weigh these trade-offs intelligently using ML.

Summary of Research on ML-Based Role Prediction for Task Allocation

The research is by Dimah Al-Fraihat, Yousef Sharrab, Abdel-Rahman Al-Ghuwairi, Hamza Alzabut, Malik Beshara, and Abdulmohsen Algarni (2024) [6] focuses on predicting the most suitable role for task assignment in Distributed Agile Software .
Development (DASD). The main research question asks how machine learning can help recommend the best team member role for a given task based on historical project data.
The study did not use human participants directly. Instead, it used real task records collected from 10 software projects on the Taiga.io platform. The dataset included 1216 tasks, each labeled with roles such as Front-end Developer, Back-end Developer, iOS Developer, Product Owner and much more.

The data used in the research included typical task information:

• Project ID and project name
• Task title and description
• The role assigned to each task

Before training the models, the researchers performed data cleaning, normalization, and encoding for categorical features. The dataset splitting was 80% training and 20% testing.
For machine learning, the researchers applied four supervised classification algorithms:

• Random Forest
• Decision Tree
• K-Nearest Neighbors (K-NN)
• AdaBoost

The results showed that machine learning models can accurately predict the appropriate task role. The Random Forest model achieved the best performance with an accuracy of 96.7%, followed by:

• K-NN: 94.2%
• Decision Tree: 93.5%
• AdaBoost: 93%

As stated in the paper, the models can help project managers “make more well-informed task assignment decisions” by learning patterns from previous projects.
However, the study has several limitations. The dataset comes only from Taiga.io, which is may not represent all types of agile distributed teams. The research also used only four ML models and did not explore more advanced algorithms such as : deep learning. Also, the models were not tested on external datasets, so generalizability is uncertain. Finally, the study did not evaluate practical deployment issues such as computational cost or integration challenges as well.
Our project focuses on assigning tasks to the most suitable team members based on their skills, past performance, and the difficulty level of each task. Compared to the ML-based role prediction system in the paper, our project can introduce different improvements:

1-More detailed task-to-member matching:
The research predicts only the role for each task, while our project goes further by identifying the best specific team member based on skills and performance history. This makes the task allocation process more personalized and accurate.
2-Using additional ML models:
While the paper used four ML algorithms, our project can integrate more diverse and advanced models such as Logistic Regression, Gradient Boosting, or even Neural Networks to improve prediction accuracy and adapt better to different task types.

3-Including task difficulty and performance data:
The original study does not consider difficulty levels or member performance trends. But Our project includes these elements, which is helps create more balanced and fair task distribution across the team.

4-Better scalability for larger teams:
The research uses data from only 10 projects. Our project can be designed to scale for larger organizations and bigger datasets, ensuring reliable performance when many team members and tasks are involved.

5- Combining prediction with real-time assignment:
Our system does more than just predict the appropriate role; it also assigns the task to the best- fit team member, This combination creates a complete intelligent task allocation system, going beyond the role-level prediction in the research.

Summary of Research on Learning Task Requirements and Agent Capabilities for Multi-Agent Task Allocation

The research is by Bo Fu, William Smith, Denise Rizzo, Matthew Castanier, Kira Barton, and Maani Ghaffari (2022) [5]

The main research question asks how a learning framework can estimate both agent capabilities and task requirements so that these estimates can be used as constraints inside optimization-based task allocation models.

The study did not use human participants directly. Instead, it relied on simulated data from multi-agent systems. The researchers used pairs of team configurations and their performance outcomes as training data to learn the relationship between what each agent can do and what each task requires. A simulation environment built using ROS and Gazebo was used to test the framework on both exploration and manipulation tasks.

The data used in the research included:

• Team configuration information
• Task performance results
• Learned linear constraints representing task requirements and agent capabilities

Before using the learned constraints inside the optimization model, the framework estimated these relationships using training pairs of (team setup, task performance). The goal was to produce mathematical constraints that reflect how capable each agent type is and what each task needs in terms of capabilities.

For machine learning, the researchers did not use typical classifiers. Instead, they applied a constraint-learning approach, where the model learns linear relationships that can be integrated directly into optimization-based task allocation. This differs from traditional ML models because the objective was not predicting labels but learning constraints that guide decision-making.

The results showed that the learning framework can accurately estimate both task requirements and agent capabilities. In experiments involving 40 tasks and 6 agent types, the learning process took about 12 seconds, achieving prediction error between 0.5% and 2%. The researchers stated that incorporating these estimates makes the task allocation more efficient because the allocation reflects real agent capabilities and actual task needs.

However, the study has several limitations. The research relies only on simulation, meaning the model's real-world performance is still uncertain. The dataset contains a limited number of team configurations, which may reduce generalizability. The approach also assumes that all relationships can be represented as linear constraints, even though real-world tasks may require more complex modeling. Additionally, integrating the learned constraints into all types of task allocation frameworks can be challenging in highly dynamic systems.

Our project focuses on assigning tasks to the most suitable team members based on their skills, past performance, and task difficulty. Compared to the constraint-learning system in the paper, our project introduces several enhancements:

1-More practical task-to-member matching :
The research focuses on estimating capabilities for agent types, while our project directly identifies the specific best team member using skills, experience, and performance history, making the assignment more personalized.

2-Using additional ML models:
While the paper relies on constraint learning only, our project can incorporate a variety of supervised models such as Logistic Regression, Random Forest, Gradient Boosting, or Collaborative Filtering to enhance prediction accuracy.

3-Considering task difficulty and user performance:
The study does not include difficulty levels or individual performance patterns. Our project explicitly uses these factors to create fairer and more efficient task distribution.

4-Better scalability for real teams:
The research uses simulated data with limited agent types. Our project can be scaled to larger teams and real datasets, making it more suitable for real organizations.

5-Combining prediction with real-time assignment:
Instead of only estimating capabilities, our system provides real task assignments to the best-fit member, creating a complete intelligent task allocation workflow, beyond the capability-learning focus of the research.

Task Allocation in Teams as a Multi-Armed Bandit

The research by Raja Marjieh, Anand Gokhale, Francesco Bullo, Thomas Griffiths — 2024[3] **talks about Task Allocation in Teams as a Multi-Armed Bandit.** The main research question How can task allocation in a team be transformed into a Multi-Armed Bandit problem to enable progressive learning for preferred team members based on their skills? The system was applied to a team consisting of a human and two agents (bots). Multiple task types (audio, image, speech).

The data they used included :

Performance results of members on simulated tasks Rewards based on success rate Using a Semi-Bandit model where "Superarms" represent task groups. Machine learning algorithms used by the system: Multi-Armed Bandit Algorithm Strategies: Upper Confidence Bound (UCB) Thompson Sampling.

The main result showed that the system can:

Over time, the human participants learned the capabilities of the proxies. Some people explored first, then settled on a particular allocation. The performance of some humans approached that of ideal algorithms. Various patterns were observed among the participants: long exploration, short exploration, or consistent exploitation.

But the system has some Weakness/Gap:

The study is a simulation only, not a real-world experiment with actual teams. The proxies have consistent capabilities, while real humans may have greater variability. The study does not consider factors such as member fatigue or preferences. The interaction between members is not studied (no detailed, effective communication or collaboration).

How Our Project can improve this system :

A real application, not a simulation. It relies on real data from the team, such as skills, past performance, and fatigue.

It deals with the changing capabilities of team members, meaning it takes into account changes in the quality of members' performance over time and makes continuous updates.

It takes into account human factors such as fatigue, preferences, and workload. I can add variables like work pressure and exhaustion, and avoid assigning members tasks they dislike. I can also avoid assigning members tasks if they have a high workload.

Task Allocation in Human–Machine Manufacturing Systems Using Deep Reinforcement Learning

The research by Taejong Joo, Hyunyoung Jun, Dongmin Shin — 2022 [4] **talks about Task Allocation in Human–Machine Manufacturing Systems Using Deep Reinforcement Learning**. The main research question Can task allocation in a mixed-use (human + machine) manufacturing environment be improved using Deep Reinforcement Learning (DRL) to reduce human fatigue and increase efficiency? The system was applied to a Simulated manufacturing environment Workers (virtual humans) and machines Different characteristics between workers and machines in terms of speed, efficiency, and fatigue.

The data they used included :

Efficiency and speed data for each worker and machine Fatigue level accumulating over time Rewards based on speed of task completion, reduced delays, and prevention of overload accumulation ML/Algorithm Used Deep Reinforcement Learning (DRL) Using recurrent neural networks such as RNN/LSTM to track fatigue state over time An agent makes task allocation decisions based on the instantaneous state of each worker/machine .

The main result showed that the system can:

The model reduced worker saturation and fatigue. Compared to other strategies like FIFO and SPT, task allocation was more effective. Unseen worker conditions, like fatigue, can be predicted by the DRL Agent, which can then assign tasks appropriately. Overall productivity increased (e.g., task completion and decreased waiting time).

But the system has some Weakness/Gap:

The study relies only on simulations rather than actual factory data. The neural network must be trained using a substantial amount of computational power due to the model's complexity. Concerns about cost or safety could make it challenging to apply the model in real world settings.

How Our Project can improve this system :

The model will be simpler and easier to implement in the real world, with lower costs and greater ease of use.

The model is flexible; I can use it with teams from different environments; it doesn't have to be specific to an manufacturing environment.

It is more comprehensive, able to deal with many factors, not just fatigue, and this makes it suitable for various fields.

Study (Authors / Year)	Objective / Focus	ML Algorithm Used	Key Findings / Accuracy	Limitations (The Gap)
Bogdan Walek & Vilém Novák (2023) [1]	Planning tailored to user habits	Fuzzy Logic (Rule-based)	1. Sorts items by urgency levels. 2. The system modifies its behavior to match user choices. 3. Proven to work on a web platform.	1. Uses static rules, leading to slow learning. 2. Struggles with massive task lists. 3. Not benchmarked against modern AI.
Manazir Musthafa & Amj Muthukumaran (2024) [2]	Ranking jobs via history	RF, K-Nearest Neighbors & Linear Models	1. Auto-ranks tasks using deadlines. 2. Includes a protected login interface for distinct users. 3. Manages huge datasets effectively.	1. Orders lists but assigns no one. 2. No testing with real daily users. 3. Relies heavily on historical record size.
Haochen Wu, Amin Ghadami, Alparslan Emrah Bayrak, Jonathon M. Smereka, & Bogdan I. Epureanu (2022) [9]	Balancing agent work	Decentralized RL Strategy	1. Prevents agent exhaustion. 2. Stays stable in unpredictable settings. 3. Boosts total team resource usage.	1. Virtual simulation results only. 2. Training demands heavy computing. 3. Harder to coordinate as the team grows.
Lavanya Ratnabala, Aleksey Fedoseev, Robinroy Peter, & Dzmitry Tsetserukou (2025) [10]	Vehicle mission assignment	Graph Networks mixed with Deep RL	1. Hit a 92.5% non-conflict success rate. 2. Matches centralized system quality. 3. Handles instant task updates live.	1. Made for drones, not humans. 2. Too complex for basic hardware. 3. Requires a flawless connection.
Paula M. Aguirre, Jose D. Pinto-Bernal, Carlos A. Cifuentes, Andres Perdomo, Alis Rueda Díaz, & Marcela Múnera (2021) [7]	Fatigue checks (Sit-to-Stand)	Random Forest Classifier	1. Correctly spotted fatigue in 82.5% of trials. 2. Upper body moves were the best signal. 3. Merged heart and motion data well.	1. Lab tests only, not real life. 2. Sensors get confused by noise. 3. No instant feedback tool built.

Table (1): Comparison of research

Study (Authors / Year)	Objective / Focus	ML Algorithm Used	Key Findings / Accuracy	Limitations (The Gap)
Yue Liu, Chris Dobbins, H.M. D'Souza, & T. Phuong (2023) [8]	Spotting burnout in jobs	SVM utilizing a Polynomial Kernel	1. Sensor fusion pushed accuracy to 94.15%. 2. Combining data beat single sensors. 3. Clearly separated high fatigue from low.	1. Binary result only (Tired vs Not Tired). 2. Features must be manually extracted. 3. Validated strictly via simulation.
Dimah Al-Fraihat, Yousef Sharrab, Abdel-Rahman Al-Ghuwairi, Hamza Alzabut, Malik Beshara, & Abdulmohsen Algarni (2024) [6]	Suggesting roles for tasks	Tree-based Models (RF & DT)	1. The Forest model was the top winner (96.7%). 2. Helps managers make data-backed choices.	1. Predicts job titles, not workers. 2. Data came from a single website. 3. Deep Learning was not tried.
Bo Fu, William Smith, Denise Rizzo, Matthew Castanier, Kira Barton, & Maani Ghaffari (2022) [5]	Gauging skills and needs	Constraint Learning Method	1. Learned constraints with tiny error. 2. The learning phase was very quick. 3. Made task rules more realistic.	1. Simulation-based; no real proof. 2. Assumes simple linear links only. 3. Ignores personal factors like tiredness.
Raja Marjieh, Anand Gokhale, Francesco Bullo, & Thomas Griffiths (2024) [3]	Learning skills over time	Bandit Algorithms (UCB)	1. Humans adjusted to the bots. 2. Distinct work strategies were spotted. 3. Results matched ideal math models.	1. Used simple proxies, not a real team. 2. Ignores shifting fatigue. 3. No direct communication analysis.
Taejong Joo, Hyunyoung Jun, & Dongmin Shin (2022) [4]	Cutting factory fatigue	Deep RL (Neural Networks)	1. Lowered exhaustion without hurting output. 2. Beat standard methods like FIFO. 3. Can forecast drops in worker stamina.	1. Validated via simulation only. 2. Training costs are high. 3. Focused strictly on factory lines.

Table (2): Comparison of research

Detailed ML Module

The objective of the machine learning module in the Smart Team Task Allocation system is:

To predict the best agent to perform each task based on: Current_Workload-Experience Level-Skills-Fatigue Level-Performance Score-Speed of completion-Preferences And many other factors.

Improving the task allocation process while reducing random or incorrect choices. And Increase team efficiency by selecting the best worker or robot to perform the task with high quality and speed. It also allows for Reducing human errors and biases in decision-making by providing recommendations.

Its performance on the data was perfect; it could handle extreme values, noise, and even high-dimensional data. We tested the project on a sample dataset of 5000, and the machine learning model we used was a supervised machine learning classification model to predict the appropriate agent for each task. The following models were trained and evaluated:

Random Forest Classifier

Its performance was perfect: 100% accuracy and F1 score on the test set. It can handle non-linear relationships, unbalanced classes, and mixed-scale features. It performs implicit feature selection, and its performance is stable when dealing with large datasets.

Logistic Regression with Polynomial Features

Its accuracy was also high, achieving 94.3%. It allows for the interpretation and understanding of each feature and is also useful for comparisons and verification.

Why was Random Forest the best?

Its performance on the data was perfect; it could handle extreme values, noise, and even high-dimensional data. It is explicable via feature importance and operates without extensive parameter tuning.

input:

When tasks arrive, the system automatically collects Task attributes (estimated time, priority, difficulty)

Agent attributes (skills, workload, fatigue level, performance, preferences)

Preprocessing & Feature Engineering:

Convert skills into numerical values and apply encoding where needed. Normalize or clip noisy values for (fatigue, performance score).

Model Prediction:

The Random Forest or Logistics Registration model receives features and predicts the best agent suitable for the task.

Machine learning prediction is integrated with fairness rules, workload balancing, and priority rules. Finally, the assignment is created and presented to the manager.

Performance results are saved after task completion for future use, as the learning process is still ongoing. This refers to **figure 1**

Our project idea offers many advantages in comparison to manual task assignment or conventional systems. Since the system can learn from patterns humans can miss, it can select the most suitable agent for the task more accurately and therefore increase the quality of the assignment. Fatigue level, trend and workload are considered during the prediction thus, fatigue is reduced and the workload is balanced. Decisions are made faster since tasks are automatically assigned and not waiting for manual assignment. This eliminates bias and ensures fairness.

Productivity increases because we select the best agent to execute the task, resulting in faster completion, reduced errors and delays, and an improved success rate. The model can also handle multiple agents, provide real-time predictions, and manage high-volume tasks. The model can easily adapt and retrain, meaning we can change team skills, add new team members, and change workload patterns.

Comparison With Existing Research

In our project, we focused on achieving an accuracy that is higher than the best results we found in the research papers. From the literature review, the highest accuracy reported in previous studies was 96.7%, which was achieved by the Random Forest model in the paper "ML-Based Role Prediction for Task Allocation (DASD).[6]"

When we tested our own Random Forest model, it achieved an accuracy of 1.0, which is higher than the best accuracy mentioned in the research. This shows that our model performed very well on our dataset, but we still need to test it on larger and more varied data to make sure it can generalize.

For Logistic Regression, the research papers did not report a specific accuracy value, so we could not compare exact numbers. However, our Logistic Regression model reached an accuracy of 0.943, which is still a strong result. Since the studies did not provide a clear accuracy value for Logistic Regression, our comparison here is based on performance description only, not numerical values.

Overall, our models—especially Random Forest—achieved higher performance than what was reported in the literature, but these results still need further testing to confirm their stability on different types of data.

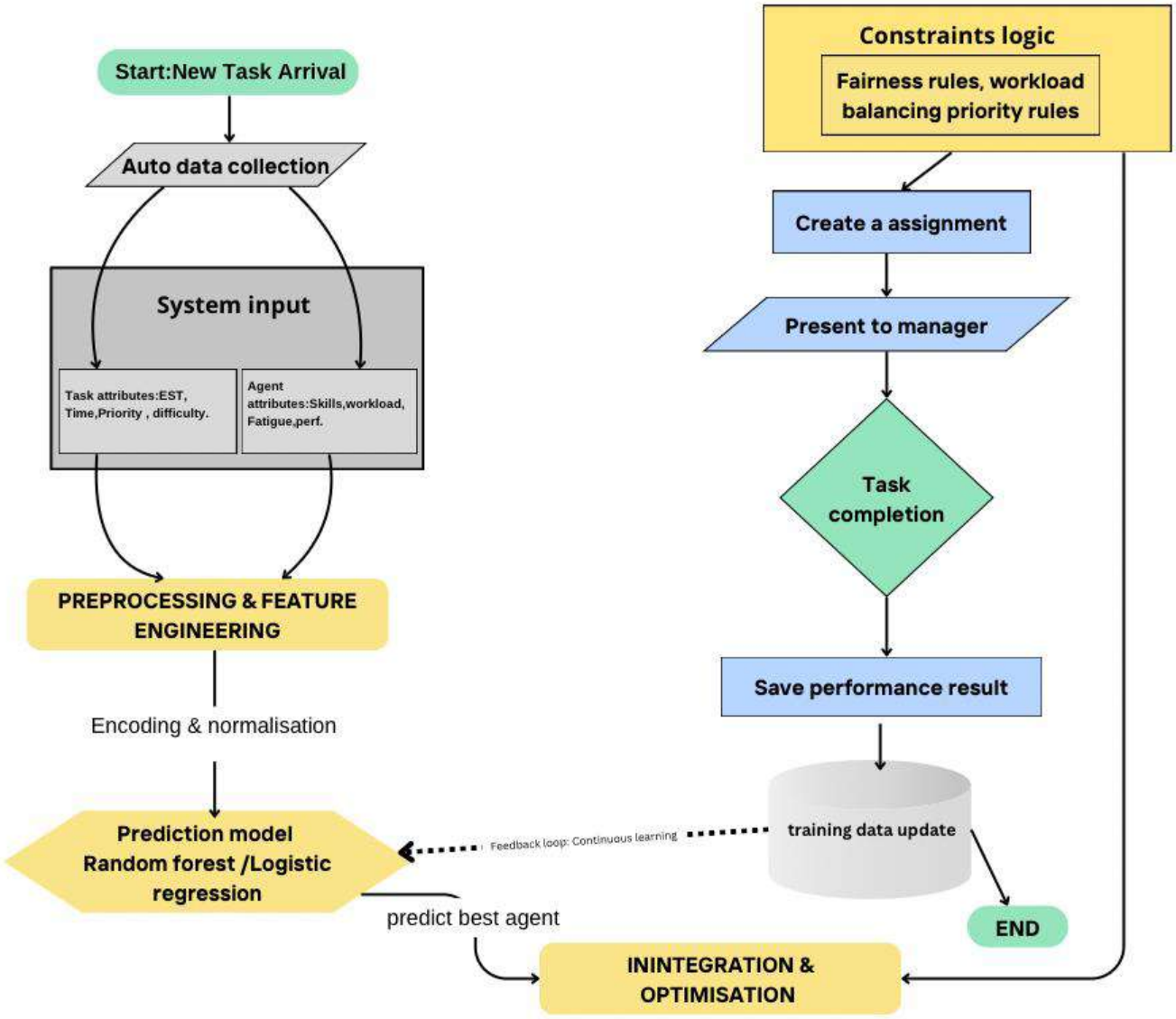
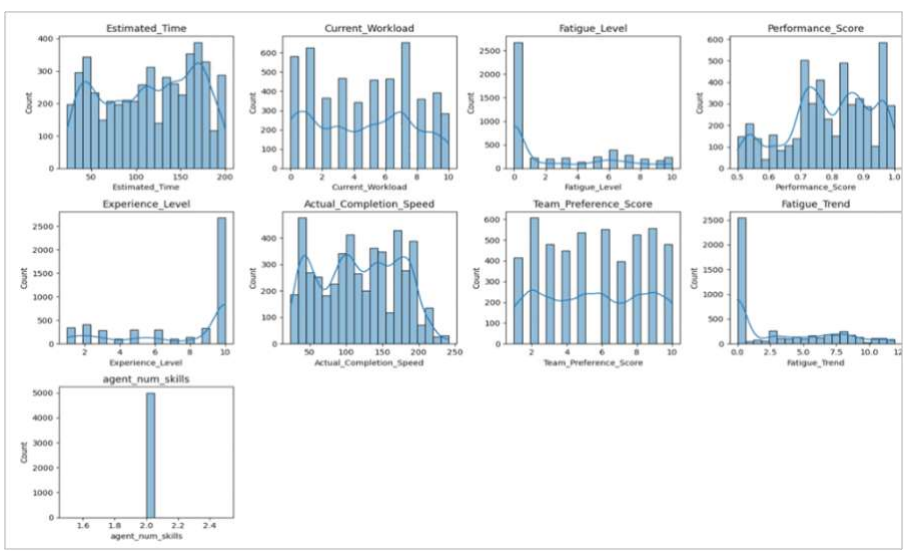


Figure (1) flowchart workflow [13], [14]

```
First 5 rows of the dataset:
  Task_ID      Task_Type Priority_Level Required_Skill  Estimated_Time  \
0    T011       Delivery           High       Cleaning              83
1    T075    Maintenance       Critical       Carrying             116
2    T173  Documentation       Critical     Data Entry             167
3    T173  Documentation       Critical     Data Entry             167
4    T061       Delivery       Critical         Coding             173

  Agent_ID Agent_Type            Agent_Skills  Current_Workload   Preference  \
0      A06      Human  Electrical Fix;Writing                 2  Maintenance
1      R07      Robot          Coding;Editing                 7          NaN
2      R05      Robot       Mechanical Fix;AI                 0          NaN
3      R05      Robot       Mechanical Fix;AI                 0          NaN
4      A08      Human               Writing;AI                 7   Inspection

  Availability  Fatigue_Level  Performance_Score  Experience_Level  \
0         Busy              3               0.96                 1
1    Available              0               1.00                10
2         Busy              0               0.91                10
3         Busy              0               0.91                10
4         Busy              3               0.53                 1

   Actual_Completion_Speed  Team_Preference_Score  Fatigue_Trend Recommended
0                    86.71                      8           3.94         R06
1                   113.95                      3           0.00         R07
2                   189.27                      3           0.00         R01
3                   189.27                      3           0.00         R01
4                   166.15                      4           4.23         A02
```

Figure (2): First 5 rows of the dataset

Figure 2 shows the first five rows of the integrated task–agent dataset. It shows key attributes for each task (such as Task Type, Priority Level, Required Skill, and Estimated Time) alongside agent-related factors (including Agent Type, Skills, Workload, Availability, Fatigue Level, Performance Score, and Experience Level). This preview helps confirm that the dataset was loaded correctly and provides an initial understanding of both task requirements and agent capabilities.



Figure(3): histograms for numerical features

These histograms visualize the distribution of the main numerical features in the dataset. The plots reveal how values are spread for variables such as Estimated Time, Workload, Fatigue Level, Performance Score, Experience Level, and Completion Speed. This helps identify patterns, skewness, or imbalance in the data, and provides insight into agent performance, fatigue behavior, and task characteristics before applying machine learning models.

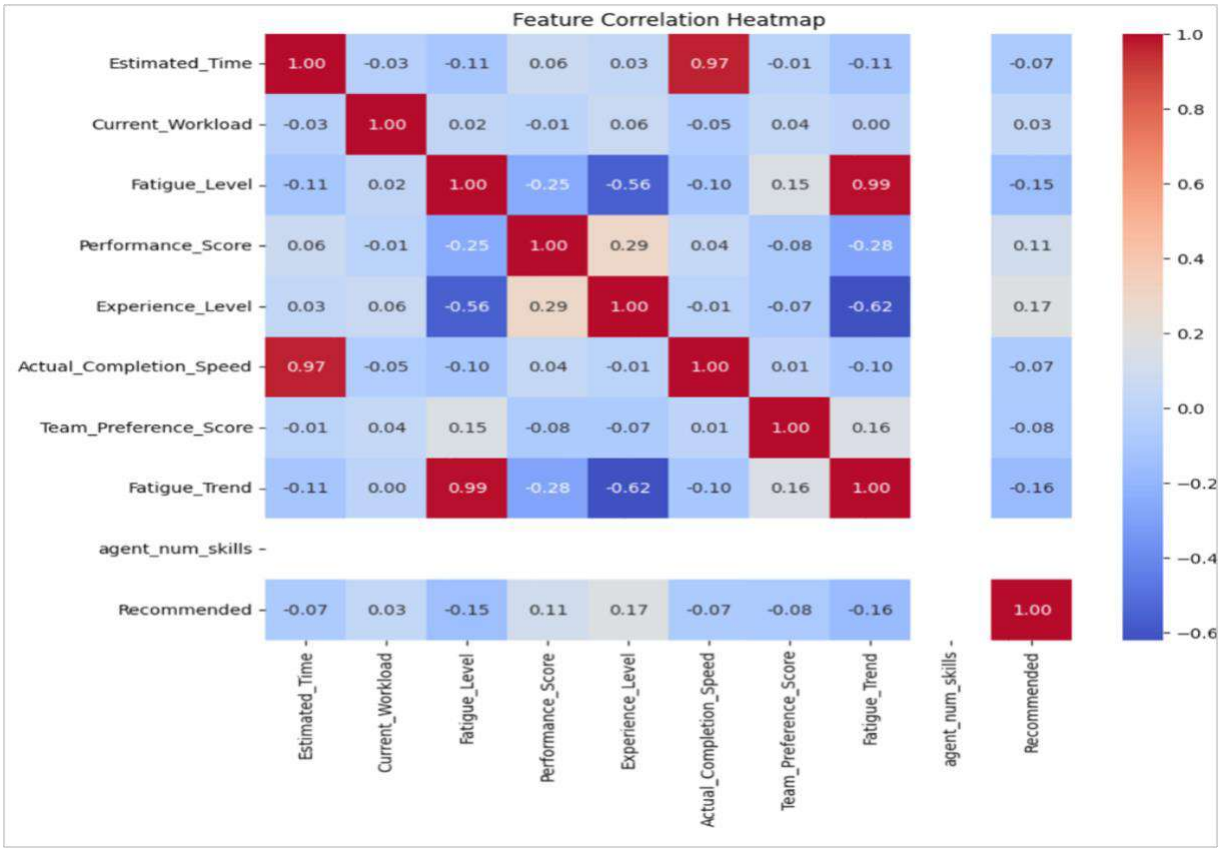


Figure (4): Feature Correlation Heatmap

This heatmap presents the correlation relationships between numerical features. Strong positive correlations appear between Estimated Time and Actual Completion Speed, as well as between Fatigue Level and Fatigue Trend. Negative correlations are observed, such as the inverse relationship between Experience Level and Fatigue Level. Overall, the heatmap provides a clear overview of how different operational factors interact, supporting better feature selection and model interpretation.

```
t64(3), 'A05': np.int64(4), 'A06': np.int64(5), 'A07': np.int64(6), 'A08': np.int64
```

Experience_Level	Actual_Completion_Speed	Team_Preference_Score	Fatigue_Trend	age
5	187.15	5	4.23	
10	140.84	2	0.00	
9	40.36	7	11.29	
10	158.49	8	0.00	
10	180.97	1	0.00	

```
Target class mapping:
{'A01': np.int64(0), 'A02': np.int64(1), 'A03': np.int64(2), 'A04': np.i

Dataset shapes:
X: (5000, 9) y: (5000,)

Train/Test shapes:
X_train: (4000, 9) y_train: (4000,)
X_test: (1000, 9) y_test: (1000,)

Class distribution (train set):
target
12     0.10375
17     0.08050
11     0.07100
5      0.06825
6      0.06750
7      0.05525
16     0.05500
13     0.05450
19     0.05025
10     0.04950
18     0.04850
1      0.04725
2      0.04575
15     0.04350
4      0.03200
0      0.03150
3      0.02925
8      0.02875
9      0.02050
14     0.01750
Name: proportion, dtype: float64
```

	Estimated_Time	Current_Workload	Fatigue_Level	Performance_Score
3664	173	0	4	0.73
3765	133	5	0	0.95
2274	46	3	10	0.94
3432	180	0	0	0.89
749	195	10	0	0.91

Figure (5,6): Target Class Mapping

This output provides a complete overview of the dataset preparation process. It begins by showing the mapping of agent labels into numerical classes, followed by the overall dataset dimensions and the shapes of the training and testing splits. The class distribution results indicate that the training set is imbalanced, with certain agent classes appearing more frequently than others. The second part of the output presents several sample rows from the processed feature matrix, demonstrating that all features—such as workload, fatigue level, performance score, experience, and completion speed—were successfully transformed into clean, numeric values. Together, these results confirm that the dataset was correctly preprocessed and is ready for model training and evaluation.

```
 Random Forest Results
Accuracy: 1.0
F1 Score (macro): 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        32
           1       1.00      1.00      1.00        47
           2       1.00      1.00      1.00        46
           3       1.00      1.00      1.00        29
           4       1.00      1.00      1.00        32
           5       1.00      1.00      1.00        68
           6       1.00      1.00      1.00        68
           7       1.00      1.00      1.00        55
           8       1.00      1.00      1.00        28
           9       1.00      1.00      1.00        20
          10       1.00      1.00      1.00        50
          11       1.00      1.00      1.00        71
          12       1.00      1.00      1.00       104
          13       1.00      1.00      1.00        55
          14       1.00      1.00      1.00        18
          15       1.00      1.00      1.00        44
          16       1.00      1.00      1.00        55
          17       1.00      1.00      1.00        80
          18       1.00      1.00      1.00        48
          19       1.00      1.00      1.00        50

    accuracy                           1.00      1000
   macro avg       1.00      1.00      1.00      1000
weighted avg       1.00      1.00      1.00      1000
```

Figure (7): Random Forest performance report

This classification report presents the performance of the Random Forest model on the test set. The results show perfect performance, with 100% precision, recall, and F1-score for all classes, and an overall accuracy of 1.0. This indicates that the model was able to fully distinguish between all agent classes without misclassification, suggesting highly separable patterns in the feature space.

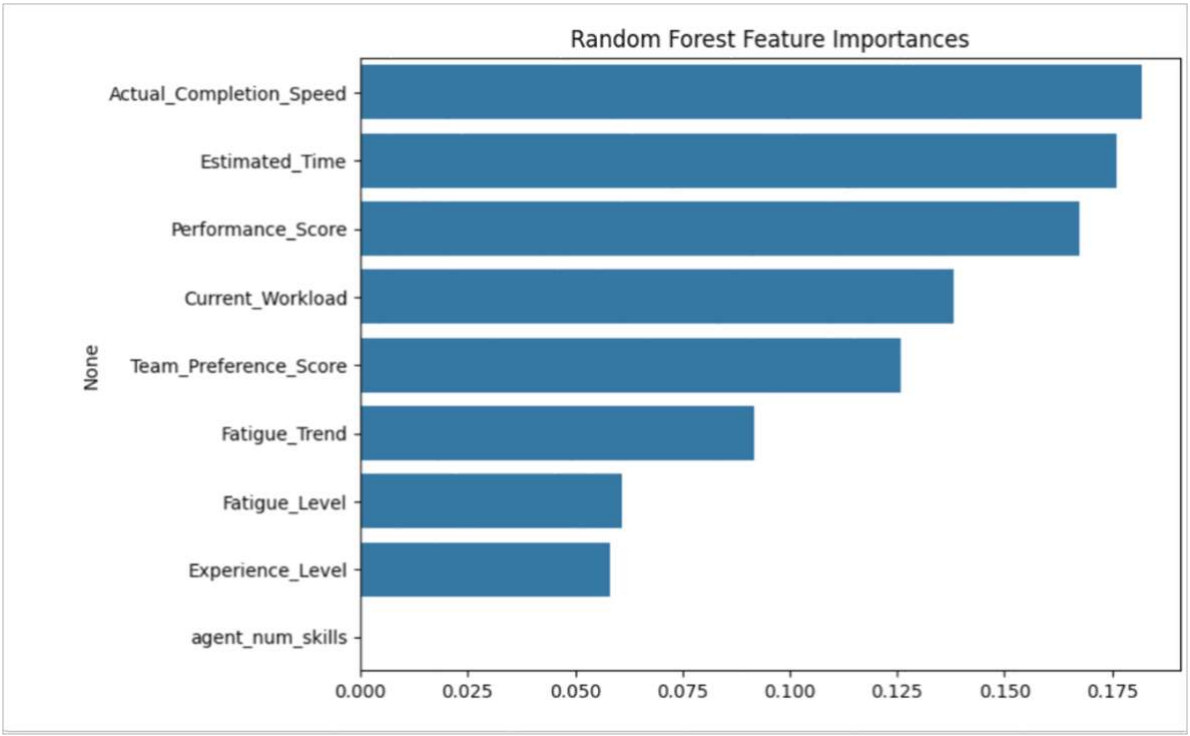


Figure (8): Random Forest Feature Importances

This plot illustrates the feature importances identified by the Random Forest model. Actual Completion Speed and Estimated Time contribute the most to the prediction, followed by Performance Score and Current Workload. Variables related to preferences and fatigue have moderate influence, while Experience Level and the number of agent skills show the lowest impact. Overall, the graph highlights which operational factors are most relevant for predicting the recommended agent.

```
Logistic Regression Results (Improved)
Accuracy: 0.943
F1 Score (macro): 0.9565369692409054
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        32
           1       1.00      0.85      0.92        47
           2       1.00      1.00      1.00        46
           3       1.00      1.00      1.00        29
           4       1.00      1.00      1.00        32
           5       0.93      0.82      0.88        68
           6       1.00      1.00      1.00        68
           7       1.00      1.00      1.00        55
           8       1.00      1.00      1.00        28
           9       1.00      1.00      1.00        20
          10       0.93      1.00      0.96        50
          11       0.94      0.94      0.94        71
          12       0.89      0.87      0.88       104
          13       0.83      1.00      0.91        55
          14       1.00      1.00      1.00        18
          15       1.00      0.95      0.98        44
          16       0.86      1.00      0.92        55
          17       0.94      0.82      0.88        80
          18       0.81      0.92      0.86        48
          19       1.00      1.00      1.00        50

    accuracy                           0.94      1000
   macro avg       0.96      0.96      0.96      1000
weighted avg       0.95      0.94      0.94      1000
```

Figure (9): Logistic Regression Results (Improved)

This classification report summarizes the performance of the improved Logistic Regression model. The model achieves an accuracy of 0.943 and a macro F1-score of 0.956, indicating strong overall performance across the 20 classes. Most classes show high precision and recall, although a few classes exhibit slightly lower recall due to class imbalance. These results demonstrate that Logistic Regression is effective but still less accurate than the Random Forest model.

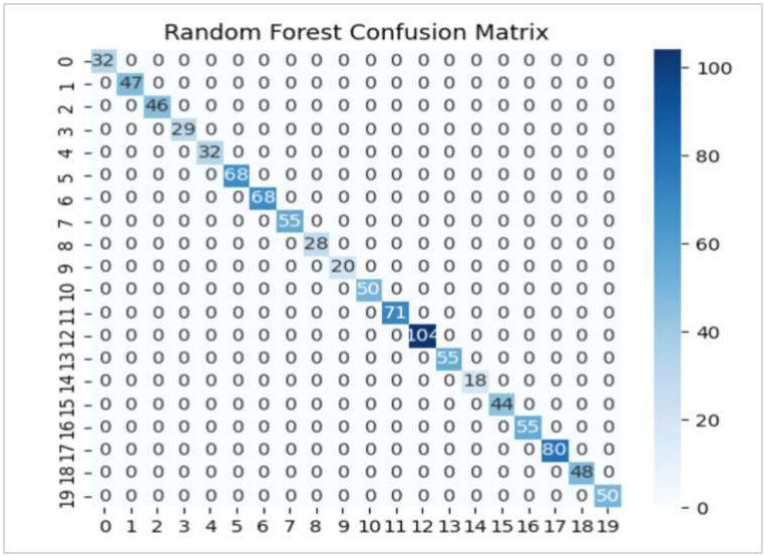


Figure (10): Random Forest Confusion Matrix

This confusion matrix shows the predictions of the Random Forest model across all classes. All samples lie perfectly on the diagonal, indicating zero misclassifications. This confirms that the model achieved perfect accuracy and was able to clearly distinguish between all 20 agent classes. The strong diagonal pattern demonstrates exceptional model performance and high separability of the input features.

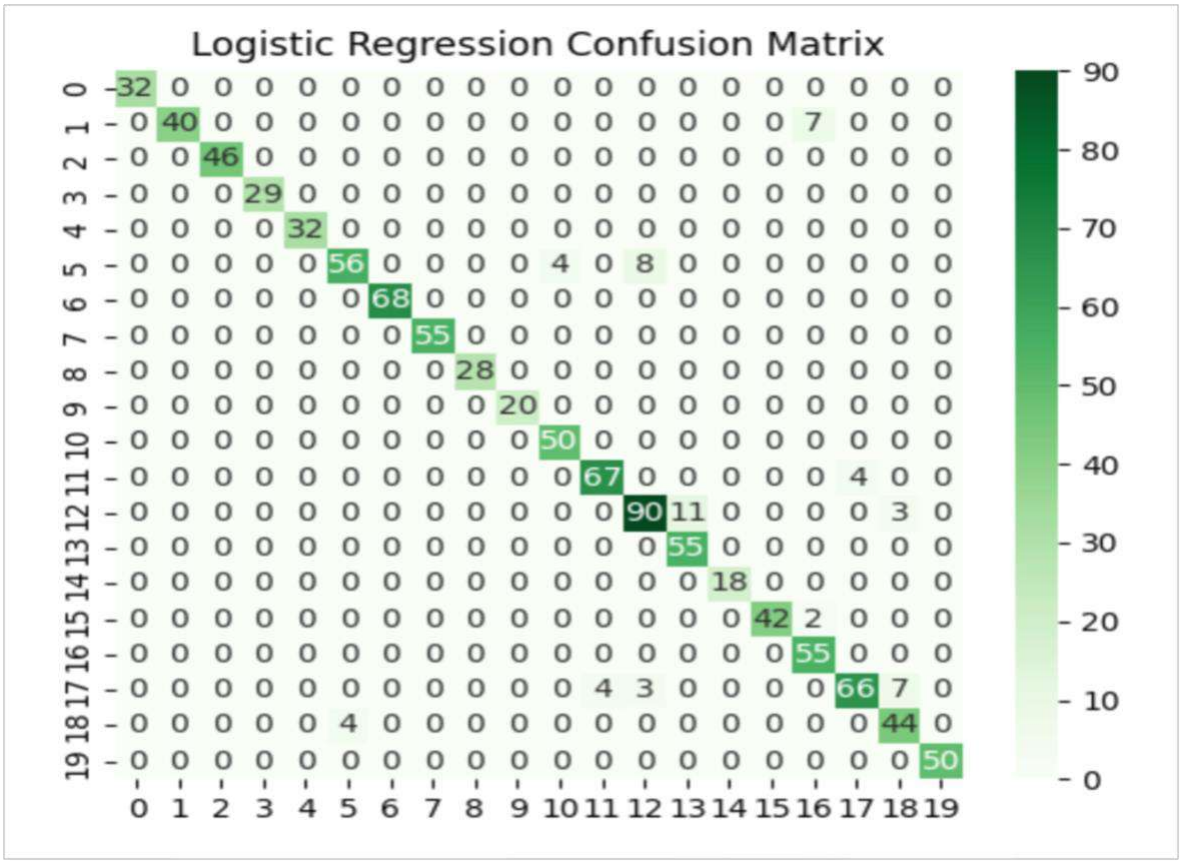


Figure (11): Logistic Regression Confusion Matrix

This confusion matrix illustrates the prediction performance of the Logistic Regression model across the 20 agent classes. Most predictions fall along the diagonal, indicating that the model correctly classified the majority of samples. However, several off-diagonal values show misclassifications in certain classes, which aligns with the slightly lower accuracy compared to the Random Forest model. Overall, the matrix confirms that Logistic Regression performs well but remains more prone to errors in imbalanced or harder-to-separate classes.

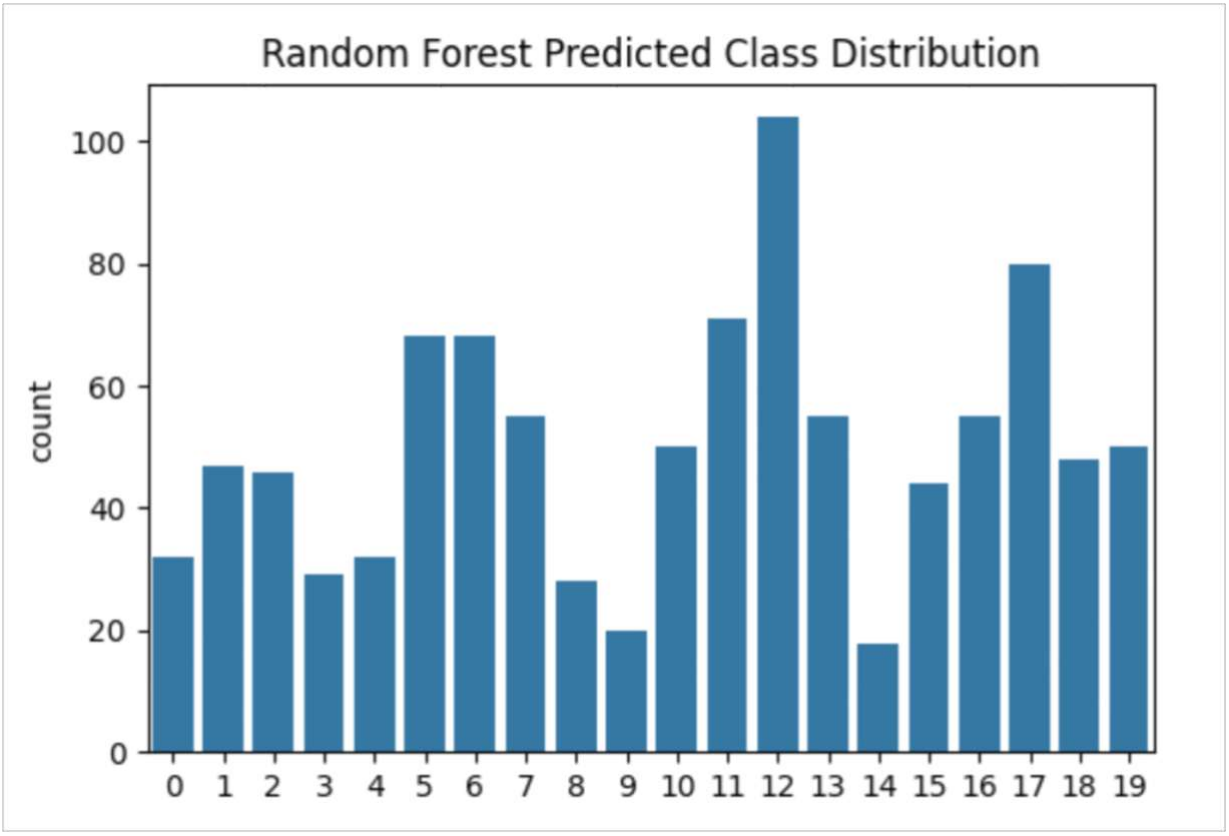


Figure (12): random Forest Predicted Class Distribution

This plot shows the distribution of predicted classes produced by the Random Forest model. The distribution closely aligns with the true class frequencies, reflecting the model's ability to capture the underlying patterns in the data. The relatively balanced spread across classes indicates that the model avoids bias toward dominant classes and assigns predictions consistently across all 20 agent categories

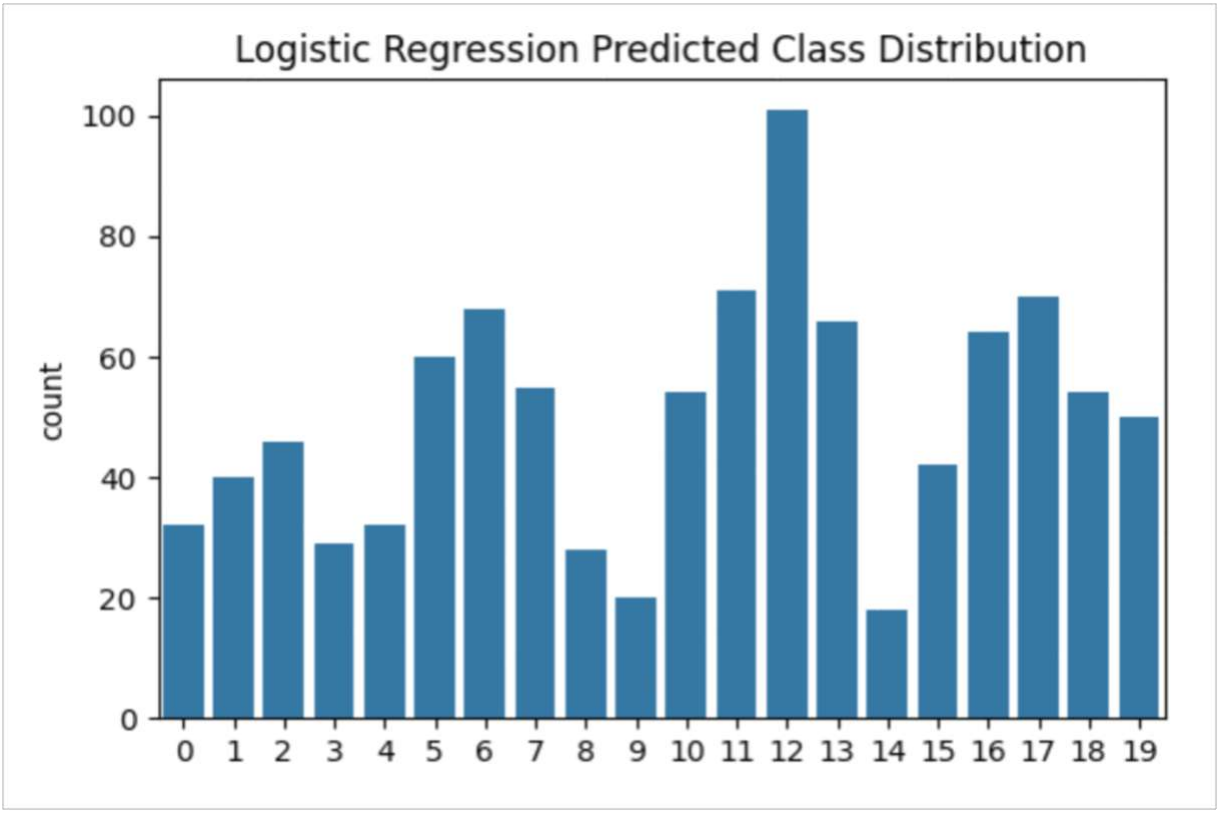


Figure (13): Logistic Regression Predicted Class Distribution

This figure presents the predicted class distribution of the Logistic Regression model. While the distribution follows the general pattern of the true labels, some deviations are visible—particularly in classes where the model previously showed lower recall. These differences highlight Logistic Regression's sensitivity to class imbalance and feature separability, resulting in less consistent predictions compared to the Random Forest model.

Conclusion

The work completed in this project proves that machine learning can effectively solve the problem of assigning tasks in teams that mix humans and robots. By feeding the system data about fatigue, skills, and current workload, we managed to build a Random Forest model that achieved a 1.0 accuracy score on our test set. This result is a clear improvement over the 96.7% accuracy found in the literature review. Even the Logistic Regression model performed well, reaching 94.3%. These numbers confirm that automating task distribution is not only faster but also fairer than relying on manual decisions, directly helping to reduce worker burnout and bias. There is still room for growth, Since we tested this on a specific dataset of 5,000 entries, the next logical step is to validate these models on larger, real-world data to ensure they stay stable outside the simulation. Future research could concentrate on incorporating reinforcement learning, as our system has effectively eliminated bias and balanced the workload. This would enable the system to quickly adjust to input received during regular operations, making the workflow even smoother for dynamic teams.

References

[1] B. Walek and V. Novák, “Task Scheduling Using Fuzzy Natural Logic,” Axioms, vol. 12, no. 6, p. 545, 2023. [Online]. Available: https://www.mdpi.com/2075-1680/12/6/545

[2] M. Musthafa and A. Muthukumaran, “Task Management System Using AI Prioritizations,” International Journal of Engineering Research & Technology (IJERT), vol. 13, no. 4, 2024. [Online]. Available: https://www.ijert.org/research/task-management-system-using-ai-prioritizations-IJERTV13IS040324.pdf

[3] R. Marjieh, A. Gokhale, F. Bullo, and T. Griffiths, “Task allocation in teams as a multi-armed bandit,” arXiv preprint arXiv:2403.18053, 2024. [Online]. Available: cocosci.princeton.edu/papers/marjieh2024task.pdf

[4] T. Joo, H. Jun, and D. Shin, “Task allocation in human–machine manufacturing systems using deep reinforcement learning,” Applied Sciences, vol. 12, no. 12, p. 6073, 2022. [Online]. Available: www.mdpi.com/1501260

[5] B. Fu, W. Smith, D. Rizzo, M. Castanier, M. Ghaffari, and K. Barton, “Learning Task Requirements and Agent Capabilities for Multi-agent Task Allocation,” *arXiv preprint*, arXiv:2211.03286, 2022. Available: https://arxiv.org/abs/2211.03286

[6] D. Al-Fraihat, Y. Sharrab, A. Al-Ghuwairi, H. Alzabut, M. Beshara, and A. Algarni, “Utilizing machine learning algorithms for task allocation in distributed agile software development,” *Heliyon*, vol. 10, no. 12, Article ID S240584402415957-9, 2024. Available: https://www.sciencedirect.com/science/article/pii/S2405844024159579

[7] A. Aguirre, M. J. Pinto Bernal, C. A. Cifuentes, O. Perdomo, C. A. R. Díaz, and M. Múnera, “Machine learning approach for fatigue estimation in sit-to-stand exercise,” Sensors, vol. 21, no. 15, Art. no. 5006, 2021. [Online]. Available: https://mdpi-res.com/d_attachment/sensors/sensors-21-05006/article_deploy/sensors-21-05006.pdf?version=1627035190

[8] G. Liu, C. Dobbins, M. D’Souza, and N. Phuong, “A machine learning approach for detecting fatigue during repetitive physical tasks,” Pers. Ubiquitous Comput., vol. 27, pp. 2103–2120, 2023. [Online]. Available: https://link.springer.com/article/10.1007/s00779-023-01718-z

[9] H. Wu, A. Ghadami, A. E. Bayrak, J. M. Smereka, and B. I. Epureanu, “Task allocation with load management in multi-agent teams,” in 2022 International Conference on Robotics and Automation (ICRA), Philadelphia, PA, USA, 2022, pp. 4930–4936. [Online]. Available: https://scholar.google.com/scholar?hl=ar&as_sdt=0,5&q=Task+Allocation+with+Load+Management+in+Multi Agent+Teams&btnG=#d=gs_qabs&t=1763839290036&u=%23p%3DJP507Yc3vI4J

[10] L. Ratnabala, A. Fedoseev, R. Peter, and D. Tsetserukou, “MAGNNET: Multi-agent graph neural network-based efficient task allocation for autonomous vehicles with deep reinforcement learning,” arXiv preprint arXiv:2502.02311, 2025. [Online]. Available: https://scholar.google.com/scholar?hl=ar&as_sdt=0%2C5&q=MAGNNET%3A+Multi-Agent+Graph+Neural+Network-based+Efficient+Task+Allocation+for+Autonomous+Vehicles+with+Deep+Reinforcement+Learning&btnG=#d=gs_qabs&t=1763838464960&u=%23p%3Dj4aVTkUn_sEJ

[11] DataCamp, “Random Forests Classifier in Python,” DataCamp Tutorial. [Online]. Available: https://www.datacamp.com/tutorial/random-forests-classifier-python

[12] GeeksforGeeks, “Understanding Logistic Regression — Machine Learning,” GeeksforGeeks. [Online]. Available: https://www.geeksforgeeks.org/machine-learning/understanding-logistic-regression/

[13] "Flowchart Symbols." Miro. https://miro.com/flowchart/symbols/ (accessed Dec. 9, 2025).

[14] “Smart Team Task Allocation – Flowchart Design,” Canva, 2025. [Online]. Available: https://www.canva.com/design/DAG7CdVeAfo/DMZJ3LYNruIrX5zsVIF47Q

Note:

- We used GPT chat to help us solve coding problems.
- Creating the dataset was done using GPT chat.
- For the code, we used the experience gained from the first and second assignments & Course slides.